

# **Sistema IoT para Monitoramento Ambiental de Racks de Infraestrutura**

**Trabalho Prático de Internet das Coisas**

Matheus Maximowitz Prause

**Professor:** Marcos Augusto Campagnaro Mucelini

**Disciplina:** Internet das Coisas (IoT)

**Data de Entrega:** 03/07/2026

# Sumário

## **1. Introdução (OBRIGATÓRIO)**

## **2. Objetivos (OBRIGATÓRIO)**

2.1 Objetivo Geral

2.2 Objetivos Específicos

## **3. Requisitos Técnicos (OBRIGATÓRIO)**

3.1 Hardware

3.2 Software

## **4. Arquitetura do Sistema (OPCIONAL)**

## **5. Desenvolvimento (OBRIGATÓRIO)**

5.1 Esquemático do Circuito

5.2 Código Fonte (OBRIGATÓRIO)

## **6. Testes e Resultados (OBRIGATÓRIO)**

6.1 Procedimentos de Teste

6.2 Resultados Obtidos

6.3 Problemas Encontrados e Soluções

# 1 Introdução

Este projeto propõe uma solução IoT para monitorar as condições ambientais de racks de servidores e infraestrutura de TI. O problema central que estamos resolvendo é o superaquecimento e a degradação silenciosa de componentes de hardware devido a falhas no sistema de climatização do ambiente. Este problema é crítico pois a indisponibilidade de servidores gera paradas nos serviços e consertos de alto custo, sendo muito mais eficiente tratar a causa raiz antes do equipamento apresentar falha térmica. A solução proposta é um sistema embarcado de processamento local que coleta métricas ambientais e transmite alertas em tempo real caso os níveis de segurança sejam ultrapassados.

## 2 Objetivos

### 2.1 Objetivo Geral

Desenvolver um sistema de monitoramento de temperatura, umidade e qualidade do ar para ambientes de infraestrutura de TI com alertas automatizados.

### 2.2 Objetivos Específicos

- Coletar dados de temperatura e umidade a cada 5 minutos.
- Implementar comunicação Wi-Fi para envio de dados através de processamento local.
- Criar dashboard web para visualização em tempo real do estado do hardware.
- Documentar a solução e disponibilizar o código em um repositório no GitHub para compor o portfólio no LinkedIn.

## 3 Requisitos Técnicos

Os componentes utilizados na montagem da solução incluem:

### 3.1 Hardware

- **Microcontrolador:** ESP32 DevKit V1
- **Sensores:** DHT22 (temperatura/umidade), MQ-135 (qualidade do ar)
- **Atuadores:** LED RGB (indicação visual), Buzzer (alertas sonoros locais)
- **Componentes:** Protoboard 400 pontos, Resistores 330Ω, Capacitor 100nF, Jumpers, Fonte 5V

### 3.2 Software

- **IDE:** Arduino IDE 2.0

- **Linguagem:** C++
- **Protocolos:** MQTT via Wi-Fi
- **Plataforma:** Adafruit IO + ThingSpeak
- **Bibliotecas:** DHT.h, PubSubClient.h, WiFi.h

## 4 Arquitetura do Sistema

**Descrição do Fluxo:** Os sensores DHT22 e MQ-135 coletam os dados do rack que são lidos pelo ESP32 a cada 5 minutos. O microcontrolador processa os valores e publica via MQTT no tópico 'infra/rack01/dados'. O Adafruit IO recebe os dados e exibe em dashboard, além de enviar alertas por e-mail quando os valores ultrapassam limites.

- **Arquitetura da Rede:** O ESP32 conecta-se à rede sem fio na faixa 2.4GHz, transmitindo pacotes TCP/IP direcionados ao broker MQTT hospedado em nuvem utilizando a porta padrão 1883.

## 5 Desenvolvimento

### 5.1 Esquemático do Circuito

O diagnóstico da pinagem foi executado e mapeado da seguinte forma:

- **DHT22:** VCC → 3.3V, GND → GND, DATA → GPIO 4
- **MQ-135:** VCC → 5V, GND → GND, A0 → GPIO 34
- **Buzzer/LED:** Anodos nos GPIOs 12 e 14 protegidos por resistores.

## 5.2 Código Fonte

Abaixo o escopo primário do sistema, responsável por inicializar conexões e processar o envio.

```
// Leitura dos sensores de ambiente do Rack e publicacao MQTT [cite: 47]

#include <WiFi.h> // [cite: 48]
#include <PubSubClient.h> // [cite: 49]
#include "DHT.h"

#define DHTPIN 4
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

void setup() { // [cite: 51]
  Serial.begin(115200); // [cite: 51, 53]
  // Inicialize seus componentes [cite: 55]
  dht.begin();
  setup_wifi();
  client.setServer(mqtt_server, 1883);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();

  float t = dht.readTemperature();
  float h = dht.readHumidity();

  if (!isnan(t) && !isnan(h)) {
    client.publish("infra/rack01/temp", String(t).c_str());
  }

  delay(300000); // Coleta a cada 5 minutos
}
```

- **Repositório com Código Completo:** [Substitua pelo LINK DO SEU REPOSITÓRIO NO GITHUB]

## 6 Testes e Resultados

### 6.1 Procedimentos de Teste

Deixei o sistema operando por 72h em testes de bancada, coletando dados a cada 5 min. Testes de estresse foram feitos expondo os sensores a condições extremas utilizando uma fonte de calor controlada para forçar o acionamento do atuador sonoro.

### 6.2 Resultados Obtidos

O sistema manteve 98% de uptime no fluxo de conectividade local com o broker. A latência média de envio aferida foi de 2.3s.

### 6.3 Problemas Encontrados e Soluções

- **Problema:** Durante o acionamento inicial, o sensor DHT22 retornava valores erráticos.
- **Solução:** Através da análise de componentes eletrônicos, foi adicionado um capacitor de 100nF entre o VCC e o GND do sensor para estabilizar o ruído lógico, sanando a ocorrência.